A PROJECT REPORT

ON

"FYSEARCH: AN OFFLINE MULTIMODAL FORENSIC SEARCH ENGINE"

SUBMITTED TO

KIIT DEEMED TO BE UNIVERSITY
IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD
OF
BACHELOR'S DEGREE IN COMPUTER SCIENCE AND ENGINEERING
BY

RACHIT PANDEY	2205916
ANWAYA ANUPRASH BISWAL	22053667
RAJUL GOEL	22053970
SANCHIT ROUT	22053979

UNDER THE GUIDANCE OF DR. SARITA TRIPATHY

SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA -751024

March 2026

KIIT DEEMED TO BE UNIVERSITY
SCHOOL OF COMPUTER ENGINEERING BHUBANESWAR, ODISHA 751024

CERTIFICATE

THIS IS TO CERTIFY THAT THE PROJECT ENTITLED

"FYSEARCH: AN OFFLINE MULTIMODAL FORENSIC SEARCH ENGINE"

SUBMITTED BY:

RACHIT PANDEY	2205916
ANWAYA ANUPRASH BISWAL	22053667
RAJUL GOEL	22053970
SANCHIT ROUT	22053979

IS A RECORD OF BONAFIDE WORK CARRIED OUT BY THEM, IN THE PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF DEGREE OF BACHELOR OF ENGINEERING (COMPUTER SCIENCE & ENGINEERING)AT KIIT DEEMED TO BE UNIVERSITY, BHUBANESWAR. THIS WORK IS DONE DURING THE YEAR 2025-2026, UNDER OUR GUIDANCE.

DATE:1/04/2026

(DR. SARITA TRIPATHY)
PROJECT GUIDE

Acknowledgements

We are profoundly grateful to Dr. Sarita Tripathy for their expert guidance, unwavering support, and continuous encouragement throughout the course of this project. Their invaluable insights, constructive feedback, and expertise have played a crucial role in shaping our research and ensuring its success. Their dedication and mentorship have provided us with the necessary knowledge, skills, and motivation to tackle the challenges encountered during the project. Without their constant motivation and belief in our capabilities, this work would not have been possible.

We also extend our heartfelt gratitude to the esteemed faculty members of the School of Computer Engineering, KIIT, for their assistance, guidance, and invaluable advice during various stages of the project. Their support, resources, and technical expertise have significantly contributed to our learning and the overall development of this work. Their encouragement and willingness to provide valuable insights have been instrumental in refining our approach and enhancing the quality of our research.

Additionally, we would like to express our appreciation to our peers and colleagues for their encouragement, discussions, and constructive suggestions, which have enriched our understanding and helped refine our approach. The collaborative discussions and exchange of ideas have played a pivotal role in broadening our perspective and improving our analytical thinking. We are especially grateful for the thought-provoking conversations and collective problem-solving efforts that have enhanced the depth and rigor of our research.

Furthermore, we acknowledge the contributions of all those who have directly or indirectly supported us in completing this project successfully. The assistance received from various individuals, whether in the form of technical help, moral support, or valuable resources, has been instrumental in the smooth execution of this project. We recognize and appreciate the contributions of researchers, developers, and domain experts whose work has laid the foundation for our research.

We sincerely appreciate each and every individual who has played a role in this endeavor, and we are grateful for the collective efforts that have contributed to the successful completion of this project.

RACHIT PANDEY
ANWAYA ANUPRASH BISWAL
RAJUL GOEL
SANCHIT ROUT

ABSTRACT

The increasing volume and diversity of digital data have made efficient information retrieval a complex task, particularly when dealing with unstructured and heterogeneous file formats. Conventional search systems based on keyword matching are limited in their ability to interpret user intent and cannot effectively process content embedded in images or scanned documents. This project presents FYSearch, an offline multimodal forensic search engine designed to enable context-aware retrieval across text, PDF, and image data. The system employs Optical Character Recognition (OCR) to extract textual content from non-editable sources and utilizes transformer-based models to generate semantic vector representations of both text and visual data. These representations are indexed using FAISS to support rapid similarity-based retrieval. By comparing query embeddings with stored vectors, the system returns results based on contextual relevance rather than exact term matching. Designed for offline operation, the system ensures data confidentiality and is suitable for secure environments with restricted network access. The architecture includes modules for document ingestion, content extraction, embedding generation, vector indexing, and user interaction. The proposed approach improves retrieval accuracy and efficiency, making it applicable to domains such as digital forensics, research data management, and secure document analysis.

Keywords: Multimodal Retrieval, OCR, Semantic Embeddings, FAISS, Offline Search, Document Analysis

Table of Contents

"FYSearch: An Offline Multimodal Forensic Search Engine 2
Acknowledgements 3
ABSTRACT 4
Contents 4
Chapter 1 Introduction 6
1.1 Background 7
1.2 Motivation 7
1.3 Problem Statement 7
1.4 Objectives of the Project 7
1.5 Scope of the Project 8
1.6 Significance of the Study 8
1.7 Overview of the Report 9
Chapter 2 Basic Concepts/ Literature Review 9
2.1 Introduction 10
2.2 Existing Document Search Techniques 10
2.3 Semantic Search and Vector Embeddings: Theoretical Background 10
2.4 Related Research Works 11
2.5 Tools and Technologies Used 12
2.5.1 Python Programming Language 12
2.5.2 Sentence Transformers (SBERT) 12
2.5.3 CLIP Model 12
2.5.4 FAISS Vector Database 12
2.5.5 Flask Framework 12
2.5.6 Data Storage and Processing(PDF/Text/Image) 12
2.6 Summary 12
Chapter 3 Requirement Specifications 13
3.1 Introduction 13
3.2 Problem Definition 13
3.3 Objectives 13
3.4 Functional Requirements 13
3.5 Non-Functional Requirements 14
3.6 System Requirements 14
3.6.1 Hardware Requirements 14
3.6.2 Software Requirements 15
3.7 Design Constraints and Assumptions 15
3.8 Project Planning and Development Approach 15
3.9 Summary 16
Chapter 4 Implementation 17
4.1 Introduction 17
4.2 System Architecture 17
4.3 Design Methodology 18
4.4 Module Description 18

4.4.1 Document Ingestion Module 18
4.4.2 Text Extraction and Processing Module 18
4.4.3 Embedding Generation Module 19
4.4.4 Vector Indexing using FAISS 19
4.4.5 Semantic Search and Retrieval 19
4.5 Data Flow Diagram 20
4.6 Implementation Environment 21
4.7 Deployment Flexibility 21
4.8 Summary 21
Chapter 5 Testing and Results 22
5.1 Introduction 23
5.2 Testing Methodology 23
5.3 Test Environment 23
5.4 Unit Testing 23
5.5 Integration Testing 25
5.6 System Testing 25
5.7 Performance Testing 26
5.8 User Acceptance Testing (UAT) 27
5.9 Result Analysis 28
5.10 Model Comparison and Performance Evaluation 28
5.11 Limitations 30
5.12 Summary 30
Chapter 6 Conclusion and Future Scope 31
6.1 Introduction 31
6.2 Conclusion 31
6.3 Project Outcomes 31
6.4 Limitations 31
6.5 Future Scope 32
6.6 Summary 32
References 33
INDIVIDUAL CONTRIBUTION REPORT: 34
INDIVIDUAL CONTRIBUTION REPORT: 35
INDIVIDUAL CONTRIBUTION REPORT: 36
INDIVIDUAL CONTRIBUTION REPORT: 37

Chapter 1 Introduction

1.1 Background

The continuous expansion of digital information across multiple domains has created new challenges in managing and retrieving useful information. Data is continuously generated in multiple formats, including text documents, images, PDFs, and other multimedia sources. This diversity makes it difficult to efficiently locate relevant information using traditional search methods. Most existing search approaches rely on exact term matching to retrieve results, where results are retrieved only if the exact terms appear in the stored data. While this approach works reasonably well for structured textual content, it becomes ineffective when dealing with unstructured formats such as scanned files, images, or handwritten records. Moreover, such systems do not interpret the intent or contextual meaning behind user queries, which often leads to incomplete or irrelevant results. Another important limitation arises in situations involving sensitive data, such as forensic analysis or confidential investigations, where internet access may be restricted for security reasons. In such environments, relying on online search tools is not feasible, highlighting the need for reliable offline solutions. Recent progress in artificial intelligence, particularly in machine learning and deep learning, has enabled more advanced approaches to information retrieval. Techniques such as semantic embeddings allow data to be represented in a way that captures contextual meaning rather than just surface-level keywords. Additionally, Optical Character Recognition (OCR) enables the extraction of textual content from images and scanned documents, making previously inaccessible data searchable.

The FYSearch system builds upon these advancements to create an offline search platform capable of handling multiple data formats. By integrating OCR, embedding models, and efficient indexing methods, the system aims to provide accurate and context-aware document retrieval.

1.2 Motivation

The development of FYSearch is driven by the practical limitations observed in existing document retrieval systems. In many real-world applications—such as digital investigations, legal analysis, and archival management—data is often stored in mixed formats, including images, scanned files, and text documents. Extracting meaningful insights from such datasets using traditional tools is both inefficient and time-consuming. Another key concern is the dependence of many modern systems on cloud-based infrastructure. While cloud solutions offer scalability, they are not always suitable for environments where data confidentiality is critical. In scenarios involving sensitive information, such as forensic investigations, it is essential that data remains within secure, isolated systems. Additionally, the growing size of datasets makes manual searching impractical. Users require tools that can quickly analyze large volumes of data and return relevant results based on meaning rather than exact wording. These challenges highlight the need for an intelligent, automated system capable of understanding content context and operating without external connectivity.

FYSearch is designed to address these needs by combining machine learning techniques with efficient search mechanisms in a fully offline environment.

1.3 Problem Statement

Existing document retrieval systems primarily rely on exact keyword matching, which limits their effectiveness when dealing with complex or unstructured data. These systems often struggle to process content stored in images or scanned documents and are unable to interpret semantic relationships between words and phrases. As a result, users may receive incomplete or irrelevant search results. Furthermore, many advanced search platforms depend on internet connectivity and

cloud services. This dependency creates challenges in secure environments where data privacy must be strictly maintained and external access is restricted.

To overcome these limitations, there is a need for a system that can:

- Extract readable text from images and scanned documents
- Interpret the contextual meaning of user queries
- Perform efficient similarity-based searches across diverse data formats
- Operate independently of internet connectivity while maintaining accuracy and speed

The FYSearch system is proposed as a solution to these challenges, offering a multimodal and context-aware approach to document retrieval.

1.4 Objectives of the Project

The main goal of this project is to develop an offline search system capable of retrieving documents based on their semantic relevance rather than simple keyword matches.

The specific objectives are:

- To design a system that can process multiple file formats, including images, PDFs, and text documents
- To implement OCR techniques for extracting textual information from non-editable sources
- To generate semantic representations of data using transformer-based models
- To apply vector-based indexing methods for efficient similarity search
- To develop an intuitive interface for user interaction and query input
- To ensure that the system functions entirely offline, preserving data security
- To enable multimodal retrieval across different types of data sources

1.5 Scope of the Project

This project focuses on creating a document retrieval system capable of handling various forms of digital data. The system supports multiple file types, allowing users to perform searches across text, images, and PDF documents using semantic understanding. Key functionalities include text extraction using OCR, generation of semantic embeddings, and indexing of data using vector-based techniques. The system is designed for offline use, making it suitable for secure environments where internet access is limited or unavailable. A simple web-based interface is also included to facilitate user interaction. However, the current implementation is intended for small to medium-scale datasets and does not address large-scale distributed deployment. Features such as advanced authentication and enterprise-level scalability are beyond the scope of this project but can be explored in future work.

1.6 Significance of the Study

This project contributes to improving how information is retrieved from large and diverse datasets. By incorporating semantic search techniques, the system enhances both the relevance and accuracy of retrieved results compared to traditional methods. A key advantage of the proposed system is its ability to function without internet connectivity, ensuring that sensitive data remains secure. This makes it particularly valuable in domains such as digital forensics, legal investigations, and government operations. The study also demonstrates how modern machine learning techniques can be applied to practical problems in information retrieval. By combining OCR, embeddings, and vector

search, the project highlights the potential of intelligent systems to improve data accessibility and analysis.

1.7 Overview of the Report

The report is organized into six chapters, each addressing a specific aspect of the project. The first chapter introduces the background, motivation, objectives, and scope of the system. The second chapter reviews existing research and explains the fundamental concepts used in the project. The third chapter outlines the system requirements and problem definition. The fourth chapter describes the system architecture and implementation details. The fifth chapter presents the testing procedures and performance evaluation. Finally, the sixth chapter summarizes the outcomes of the project and discusses potential future enhancements.

Chapter 2 Basic Concepts/ Literature Review

2.1 Introduction

The increasing availability of digital data has made efficient information retrieval an important research area. Modern datasets often include a mixture of structured and unstructured formats, which makes traditional search approaches less effective. Systems that rely only on keyword matching frequently fail to produce accurate results when the wording of a query differs from the stored content or when the data exists in non-textual forms. Recent developments in artificial intelligence have introduced new methods for handling such challenges. Techniques from natural language processing and computer vision now allow systems to interpret meaning rather than simply match terms. Methods such as embedding-based representations, text extraction from images, and similarity-based retrieval have enabled more flexible and accurate search mechanisms.

This chapter outlines the core ideas behind these approaches, reviews earlier retrieval techniques, and discusses the technologies used in the proposed system.

2.2 Existing Document Search and Retrieval Techniques

Earlier information retrieval systems were primarily designed for structured textual data and depended heavily on direct word matching. These approaches are simple but have several limitations when applied to modern datasets.

Common methods include:

- **Keyword Matching:**
  Documents are indexed based on the words they contain, and results are retrieved when those exact words appear in a query. This method does not account for variations in language, such as synonyms or paraphrased expressions.
- **Metadata-Based Retrieval:**
  Documents are categorized using attributes like titles, tags, or authors. While this can improve organization, it depends on manual labeling, which may be inconsistent or incomplete.
- **Full-Text Search:**
  The entire content of a document is analyzed during retrieval. Although more comprehensive than metadata-based methods, it still relies on literal word matching and lacks contextual understanding.

These techniques are not well suited for handling images, scanned files, or mixed data formats, which limits their usefulness in many real-world applications.

2.3 Semantic Search and Multimodal Retrieval: Theoretical Background

In this system, search results are generated by comparing contextual representations of queries with stored document data, allowing results to reflect intended meaning rather than exact wording. This is achieved by converting data into numerical representations, often referred to as embeddings, which capture relationships between words, phrases, or visual elements. In this approach, both stored data and user queries are transformed into vectors within a high-dimensional space. A similarity scoring method is used to identify how closely the query matches stored data representations. Items with higher similarity scores are considered more relevant. Multimodal retrieval extends this concept by allowing different types of data—such as text and images—to be processed together. Models designed for this purpose can generate comparable representations for both modalities, enabling cross-type

search. For example, a textual query can be used to retrieve relevant images. To handle large datasets efficiently, specialized indexing methods are used. Libraries such as FAISS provide fast search capabilities by organizing vectors in a way that supports quick comparison. Additionally, OCR plays an important role by converting text from images and scanned documents into a searchable format.

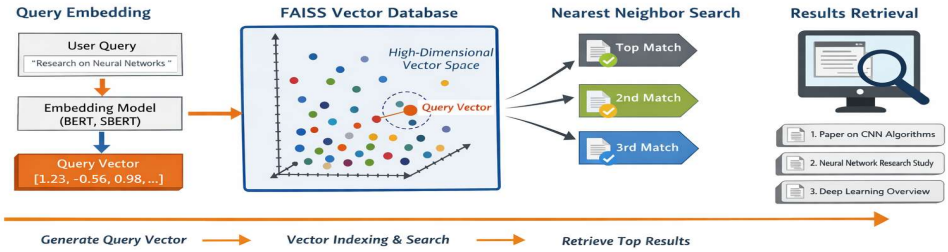

Figure 2.3: Semantic Search using Vector Embeddings and FAISS

2.4 Related Research Works

Recent advancements in information retrieval have focused on improving search accuracy through the use of machine learning and semantic representations.

- Reimers and Gurevych (2019) introduced Sentence-BERT, a modification of the BERT architecture designed to generate meaningful sentence-level embeddings. Their work demonstrated that semantic similarity tasks can be performed more efficiently using embedding-based approaches compared to traditional methods.
- Johnson et al. (2017) developed FAISS, a library optimized for large-scale similarity search. Their research showed that efficient indexing and nearest-neighbor search techniques can significantly reduce retrieval time when working with high-dimensional vector data.
- Radford et al. (2021) proposed CLIP, a model that learns a shared representation for images and text. This approach enables cross-modal retrieval, allowing systems to match visual data with textual descriptions, which is particularly useful in multimodal search applications.
- Smith (2020) emphasized the importance of Optical Character Recognition (OCR) in converting scanned documents and images into machine-readable text. This capability expands the range of searchable content and improves accessibility in document retrieval systems.
- Zhao et al. (2022) explored multimodal retrieval frameworks that combine natural language processing with computer vision techniques. Their findings indicate that integrating multiple data types leads to improved retrieval performance, especially when handling complex and heterogeneous datasets.

Overall, these studies highlight the effectiveness of combining semantic embeddings, OCR, and vector-based indexing techniques to enhance the accuracy and efficiency of modern information retrieval systems.

2.5 Tools and Technologies Used

The implementation of the proposed system relies on a combination of programming tools, machine learning frameworks, and data processing libraries.

2.5.1 Python

Python was selected for implementation because it supports seamless integration of machine learning libraries and data processing tools used in this project.

2.5.2 Optical Character Recognition (OCR)

OCR technology is used to extract text from images and scanned documents. In this system, Tesseract OCR is applied to convert visual text into machine-readable format, enabling it to be indexed and searched.

2.5.3 Sentence Transformers

Sentence Transformers are used to generate vector representations of textual data. These models capture the meaning of sentences, allowing comparison based on context rather than exact wording.

2.5.4 CLIP Model

CLIP is used to process both text and image data within a shared representation space. This enables the system to perform cross-modal retrieval, such as finding images relevant to a text query.

2.5.5 FAISS

A vector indexing library is utilized to organize and search numerical representations quickly, even when handling large datasets. It allows rapid comparison between embeddings, making it suitable for similarity-based retrieval tasks.

2.5.6 SQLite

SQLite is used for managing structured data such as document metadata and extracted content. It is lightweight and suitable for local, offline applications.

2.6 Summary

This chapter presented the fundamental concepts required for building an intelligent retrieval system. It discussed the limitations of traditional keyword-based methods and introduced modern approaches based on semantic understanding and vector representations. It also reviewed relevant research contributions and described the tools used in the system implementation. These concepts form the basis for the design and development of the proposed solution.

Chapter 3 Requirement Specifications

3.1 Introduction

This chapter outlines the requirements for developing the FYSearch system, which is designed to support intelligent document retrieval in an offline environment. The focus is on defining what the system should achieve and the conditions under which it should operate. The system is expected to handle multiple data formats, extract useful information, and provide relevant results using meaning-based search techniques. To achieve this, the system incorporates text extraction, semantic representation, and efficient indexing methods. The following sections describe the problem being addressed, the objectives of the project, and the functional and non-functional requirements needed for successful implementation.

3.2 Problem Definition

Handling large collections of digital documents has become increasingly complex due to the variety of data formats involved. Traditional search approaches are limited because they depend on exact word matching, which does not account for differences in wording or context. Another difficulty arises when information is stored in images or scanned documents, where text is not directly accessible. Without proper extraction methods, such data cannot be searched effectively. In addition, many existing systems rely on internet-based services, which may not be suitable in environments where data must remain secure and isolated.

To address these challenges, the system should be capable of:

- Extracting text from images and scanned documents
- Interpreting the meaning of user queries
- Searching across different file formats efficiently
- Operating without dependence on internet connectivity

The FYSearch system is designed to meet these requirements by combining text extraction, semantic representation, and similarity-based retrieval.

3.3 Objectives

The primary goal of this project is to create an offline system that retrieves documents based on contextual relevance rather than direct keyword matches.

The specific objectives include:

- Supporting multiple file formats such as text, PDFs, and images
- Extracting textual content from visual data using OCR
- Generating meaningful representations of data using machine learning models
- Implementing efficient search through vector-based indexing
- Providing a simple interface for user interaction
- Ensuring secure operation without external network dependency

3.4 Functional Requirements

The system is designed to perform the following operations:

- Document Ingestion: The system should be able to scan and import documents from a specified directory. Supported file formats include images, PDFs, and text documents.
- Text Extraction: The system must extract textual content from documents. For digital PDFs, text can be directly extracted, while scanned documents and images must be processed using OCR.
- Embedding Generation - The system should convert extracted text and images into semantic embeddings using machine learning models.
- Vector Indexing: The system must store generated embeddings in a vector index to enable fast similarity-based searches.
- Search Query Processing: The system should allow users to enter natural language search queries. These queries must be converted into embeddings and compared with stored document embeddings.
- Result Retrieval: The system must return the most relevant documents based on similarity scores between the query embedding and stored embeddings.
- User Interface: The system should provide a simple web interface that allows users to enter search queries and view the retrieved results.

3.5 Non-Functional Requirements

In addition to functionality, the system must satisfy the following performance and usability conditions:

- Performance: The system should retrieve relevant documents quickly even when the dataset contains a large number of files.
- Reliability: The system should consistently provide accurate search results without frequent failures or errors.
- Usability: The user interface should be simple and easy to use so that users can perform searches without requiring technical expertise.
- Security: Since the system operates offline, sensitive data remains within the local environment, ensuring a high level of data privacy.
- Scalability: The system should be capable of handling increasing volumes of documents without significant performance degradation.
- Maintainability: The system should be modular and easy to update or modify in the future.

3.6 System Requirements

The system requirements define the hardware and software environment necessary for the smooth execution of the project.

3.6.1 Hardware Requirements

- Component: Processor
- Minimum Specification: Intel Core i3 or equivalent
- Recommended Specification: Intel Core i5 or higher
- Component: RAM
- Specification: 8 GB (recommended for embedding generation)
- Storage: Minimum 20 GB available disk space
- Display: Standard monitor for viewing the web interface

- Input Devices: Keyboard and mouse
- Component: GPU
- Minimum Specification: Not mandatory (CPU mode supported)
- Recommended Specification: NVIDIA GPU (GPU support can improve embedding computation speed)

3.6.2 Software Requirements

- Operating System: Windows, Linux, or macOS
- Programming Language: Python
- Libraries and Frameworks: Sentence Transformers, FAISS, NumPy, Tesseract OCR
- Database: SQLite
- Web Interface: HTML and basic web frameworks
- Development Environment: Visual Studio Code or similar code editor
- Package Manager: pip (Python package installer)

3.7 Design Constraints and Assumptions

Several constraints and assumptions were considered during the design of the system. One of the primary constraints is the requirement for the system to operate completely offline. This means that all machine learning models and libraries must be locally available and should not depend on external cloud services. Another constraint involves hardware limitations. Since embedding generation and vector indexing require computational resources, the system must be optimized to work efficiently on standard personal computers. The system assumes that input documents are stored in supported file formats such as images, PDFs, or text files. It is also assumed that the OCR engine can accurately recognize text from scanned documents with reasonable image quality. Additionally, the system is designed primarily for small to medium-scale datasets. Handling extremely large datasets may require further optimization and more powerful hardware resources.

3.8 Project Planning and Development Approach

The project followed an iterative development model, ensuring modular design, continuous integration, and systematic testing of all system components. The development process was divided into several stages to ensure proper implementation and functionality of the proposed system.

- Requirement Analysis: Understanding the need for an intelligent document retrieval system capable of handling large collections of documents, images, and PDFs. This stage involved identifying the limitations of traditional keyword-based search systems and determining the requirements for semantic search, OCR-based text extraction, and offline operation.
- System Design: Designing the overall architecture of the system and defining individual modules such as document ingestion, text extraction using OCR, embedding generation, vector indexing, and semantic search. This stage also included selecting appropriate technologies and libraries such as Python, Sentence Transformers, FAISS, and SQLite.
- Implementation: Developing and integrating the core modules of the system using Python. This included implementing OCR-based text extraction using Tesseract, generating semantic embeddings using transformer models, indexing embeddings using FAISS, and developing a web interface to allow users to perform search queries and view results.

- Testing and Evaluation: Performing unit testing and system-level testing to ensure that each module functions correctly. The system was tested using different types of documents including images, scanned PDFs, and text files to evaluate the accuracy and efficiency of the search results.
- Deployment: Configuring the system for local offline environments and ensuring that all required models, libraries, and databases are available locally. This stage ensured that the system can operate securely without relying on internet connectivity.

This phased development approach ensured gradual enhancement of the system's capabilities while minimizing integration issues and improving the overall reliability of the proposed search system.

3.9 Summary

This chapter defined the requirements and design considerations for the FYSearch system. It described the problem being addressed, outlined the system objectives, and detailed the functional and non-functional requirements. It also specified the hardware and software environment and explained the development approach used to implement the system.

Chapter 4 Implementation

4.1 Introduction

This chapter explains how the FYSearch system was developed and converted from a conceptual design into a working application. The implementation focuses on enabling the system to accept documents, process their contents, generate meaningful representations, and return relevant results based on user queries. To achieve this, multiple components were integrated, including text extraction, representation learning, and similarity-based retrieval. The system was built using Python along with supporting libraries for handling both textual and visual data. The overall functionality is divided into independent modules, each responsible for a specific task within the processing pipeline.

4.2 System Architecture

The system architecture of FYSearch is designed to support efficient processing and retrieval of documents from large datasets. The architecture consists of several interconnected components that work together to perform document analysis and semantic search.

The main components of the system include:

1. Document Input Layer – Handles the input of documents such as images, PDFs, and text files.
2. Document Processing Layer – Extracts textual information from documents using OCR and other extraction techniques.
3. Embedding Generation Layer – Converts extracted text and images into numerical vector representations using machine learning models.
4. Vector Indexing Layer – Stores embeddings in a FAISS index for efficient similarity search.
5. Search Query Processing Layer – Processes user queries and retrieves relevant results based on vector similarity.
6. User Interface Layer – Provides an interface through which users can perform searches and view results.

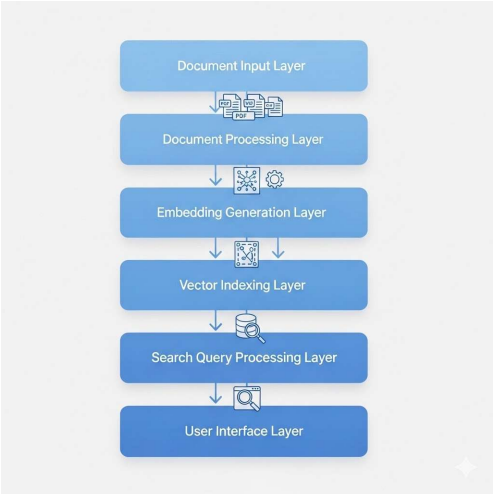

Figure 4.1: Layered Architecture of FYSearch System

This layered architecture ensures modularity and allows individual components of the system to be updated or improved independently.

4.3 Design Methodology

The system follows a structured and modular design methodology, combining both top-down analysis and bottom-up implementation to ensure scalability, maintainability, and efficient document retrieval.

- **Requirement Analysis:** Identified the need for an intelligent offline document search system capable of processing multiple data formats such as images, PDFs, and text files. Functional and non-functional requirements were defined as discussed in Chapter 3.
- **System Design:** The system architecture was divided into well-defined modules including document ingestion, text extraction, embedding generation, vector indexing, and search interface to ensure organized development and clear functionality.
- **Module Development:** Each module was developed and tested independently. Components such as OCR-based text extraction, embedding generation using transformer models, and vector indexing using FAISS were implemented and validated separately.
- **Integration and Testing:** After individual modules were completed, they were integrated to form the complete semantic search pipeline. The system was tested with different types of documents to ensure accurate retrieval and smooth interaction between modules.
- **Deployment:** The final system was configured for offline operation on local machines and secure environments. All machine learning models and libraries were stored locally to allow the system to function without internet connectivity.

The modular design ensures separation of responsibilities, where each module performs a specific task while interacting efficiently with other components of the system through a structured data processing pipeline.

4.4 Module Description

The FYSearch system is divided into several functional modules. Each module performs a specific task and contributes to the overall functionality of the system.

4.4.1 Document Ingestion Module

The document ingestion module is responsible for collecting and organizing input files within the system. It scans the input directory for supported file formats such as images, PDFs, and text documents. When a new file is detected, the system assigns it a unique identifier and stores its metadata in the database. The document is then copied to the internal storage directory for further processing. This module ensures that all documents are properly indexed and accessible to the subsequent modules in the processing pipeline.

4.4.2 Text Extraction Module

The text extraction module processes documents to extract textual information. This module supports two types of extraction:

Direct Text Extraction:
For digital PDFs and text documents, the system extracts text directly using document parsing techniques.

OCR-Based Extraction:
For scanned documents and images, the system uses Tesseract OCR to recognize and extract text from visual content.

The extracted text is stored in the database and becomes searchable by the system.

4.4.3 Embedding Generation Module

The embedding generation module converts textual and visual information into semantic embeddings. These embeddings represent the meaning of the content in a high-dimensional vector space. Text embeddings are generated using Sentence Transformer models, which capture the contextual meaning of sentences and phrases. For images, CLIP-based models generate embeddings that represent visual features. The generated embeddings allow the system to perform similarity-based searches instead of simple keyword matching.

4.4.4 Vector Indexing and Search Module

The vector indexing module is responsible for storing embeddings in a searchable index using FAISS (Facebook AI Similarity Search). FAISS enables fast similarity comparisons between vectors even when the dataset contains thousands of documents. When a user submits a search query, the system converts the query into an embedding and compares it with stored embeddings in the FAISS index. The system then retrieves the documents whose embeddings are most similar to the query. This module forms the core of the semantic search functionality in the FYSearch system.

4.4.5 Web Interface and Query Processing Module

The web interface module provides a simple user interface that allows users to interact with the system. Through this interface, users can enter search queries and view the retrieved results. The query processing component converts the user's query into a semantic embedding and sends it to the vector search module. The results are then displayed along with document metadata such as file name, location, and similarity score. This interface improves the usability of the system and allows users to perform searches without needing technical knowledge of the underlying system.

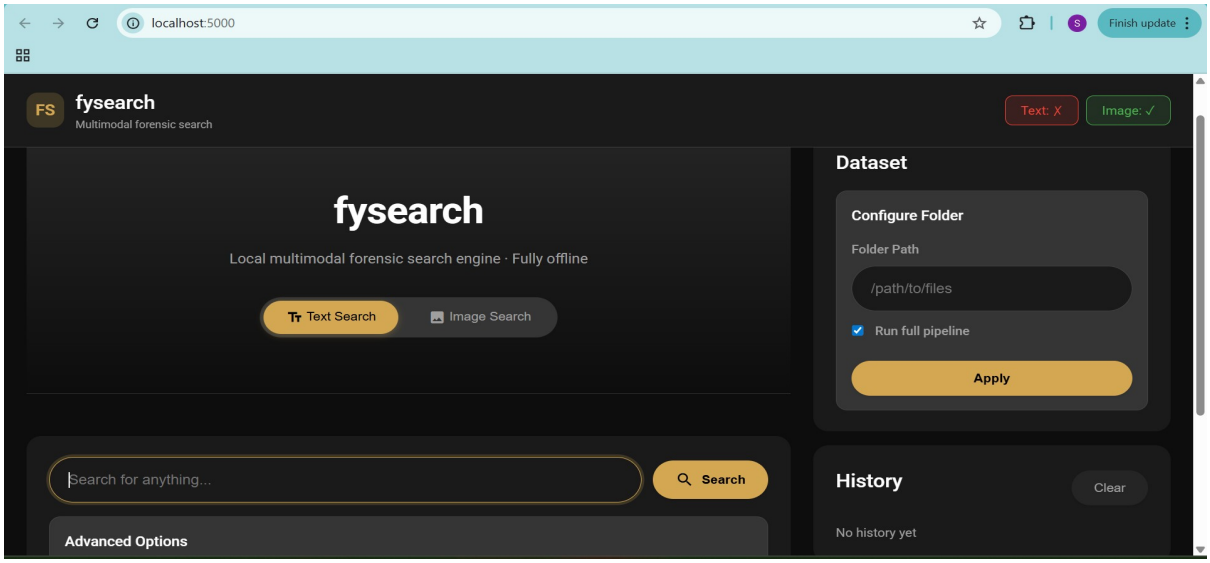

Figure 4.1: FYSearch User Interface – Home Screen

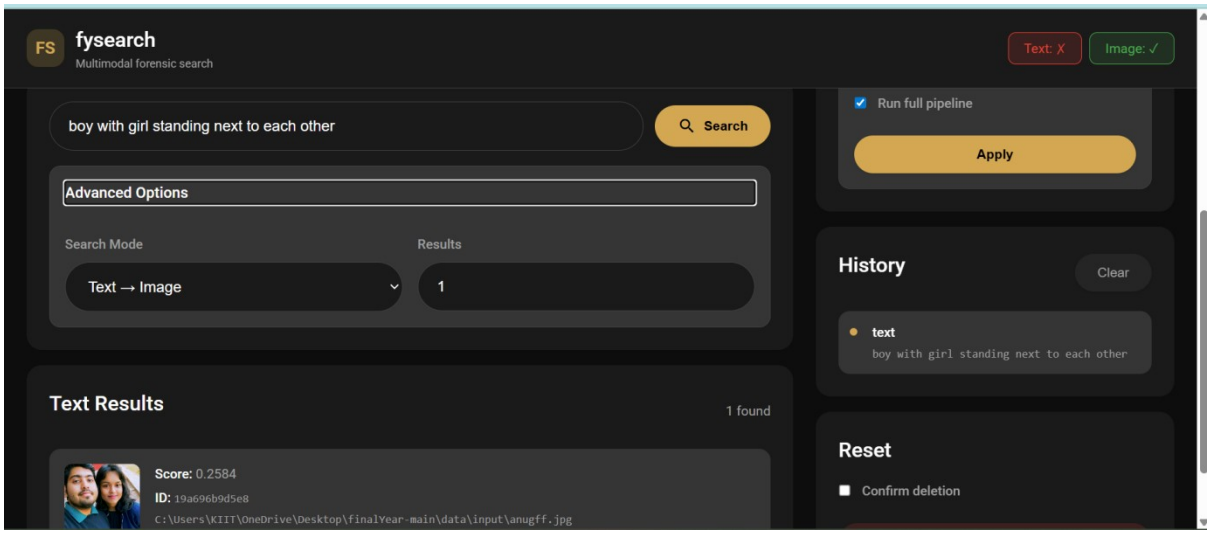

Figure 4.2: Search Results Display Interface

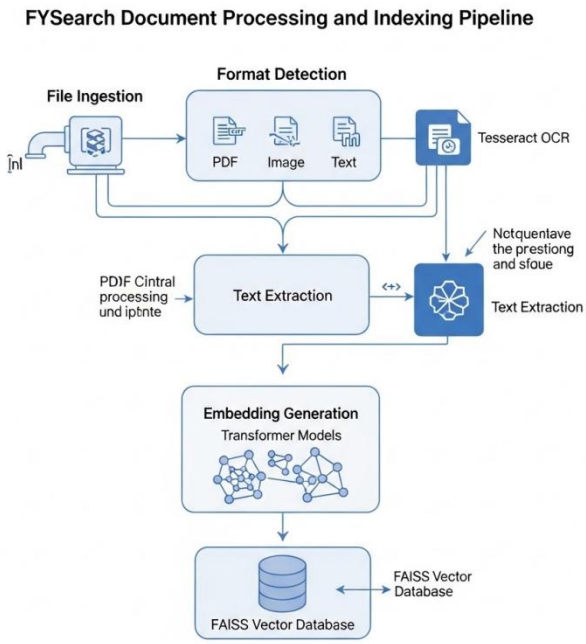

Figure 4.3: FYSearch Document Processing and Indexing Pipeline

4.5 Data Flow Diagram

The Data Flow Diagram (DFD) represents the flow of data within the FYSearch system. It illustrates how data moves between different modules and how information is processed during document ingestion and search operations.

The main stages in the data flow include:

1. Document Input – Documents are uploaded or placed in the input directory.
2. Document Processing – Text is extracted from documents using OCR or direct extraction.

3. Embedding Generation – Extracted text and images are converted into embeddings.
4. Vector Indexing – Embeddings are stored in a FAISS index.
5. User Query Input – The user enters a search query.
6. Similarity Search – The system compares the query embedding with stored embeddings.
7. Result Display – The most relevant documents are returned to the user.

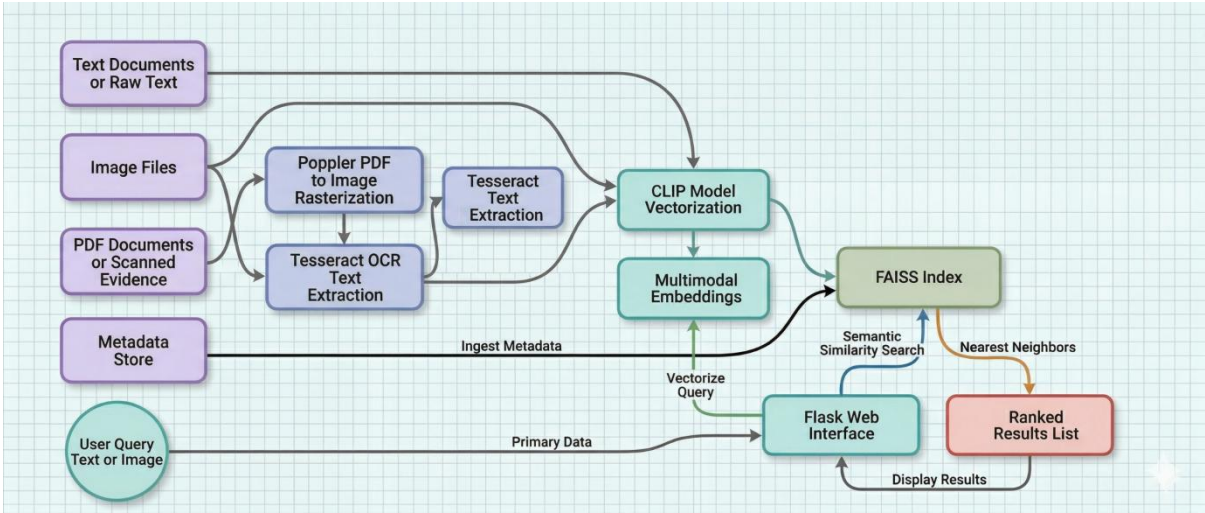

Figure 4.4: Detailed Data Flow of FYSearch System

This data flow ensures efficient processing and retrieval of documents within the system.

4.6 Implementation Environment

The FYSearch system was implemented using modern programming tools and machine learning libraries. The development environment includes:

- Programming Language: Python
- OCR Engine: Tesseract OCR
- Machine Learning Libraries: Sentence Transformers and CLIP
- Vector Search Library: FAISS
- Database: SQLite
- Web Interface: HTML-based interface
- Development Tools: Visual Studio Code

These technologies were selected due to their reliability, flexibility, and strong support for machine learning and data processing applications.

4.7 Deployment Flexibility

The system is designed to run entirely on local machines without requiring internet access. This makes it suitable for secure environments where data must remain protected. All necessary components, including models and libraries, are stored locally. The system can be deployed on personal computers or secure workstations depending on the use case. The modular design also allows future improvements, such as adding new data formats or enhancing processing techniques.

4.8 Summary

This chapter described how the FYSearch system was implemented, including its architecture, modules, and workflow. It also explained how different components interact to process documents and perform search operations. The system demonstrates an effective approach to retrieving relevant information using structured representations and similarity-based comparison..

Chapter 5 Testing and Results

5.1 Introduction

This chapter presents the evaluation of the FYSearch system to ensure that it performs as expected under different conditions. The testing process focuses on verifying that each component functions correctly and that the overall system delivers accurate and efficient search results. Different testing approaches were applied to examine individual modules, their interaction, and the performance of the complete system. The goal was to confirm that the system can process various document types, generate meaningful representations, and return relevant results based on user queries.

5.2 Testing Methodology

A structured testing methodology was followed to evaluate the system at different levels. The testing process included verification of individual modules, validation of interactions between modules, and evaluation of the complete system under real-world conditions. The testing stages included:

1. **Unit Testing:** Testing individual functions such as document loading, text extraction, and embedding generation.
2. **Integration Testing:** Ensuring proper interaction between document ingestion, embedding generation, and vector indexing modules.
3. **System Testing:** Evaluating the entire application workflow from document indexing to semantic search retrieval.
4. **Performance Testing:** Measuring system response time, resource usage, and scalability.
5. **User Acceptance Testing:** Verifying system usability and effectiveness from the user's perspective.

This structured approach ensured that the system remained reliable and stable throughout development.

5.3 Test Environment

The testing of the FYSearch system was carried out in a standard development environment with the following configuration.

Hardware Configuration

- Processor: Intel Core i5 or equivalent
- RAM: 8 GB
- Storage: 256 GB SSD
- GPU: Not required (CPU-based processing)

Software Configuration

- Operating System: Windows 10 / Linux
- Programming Language: Python
- Libraries Used: Sentence Transformers, CLIP Model, FAISS Vector Database, NumPy, Flask (for web interface)

- The testing environment simulated typical user conditions to ensure the system performs effectively without requiring specialized hardware.

5.4 Unit Testing

Unit testing focused on verifying the correctness of individual functions within the backend modules responsible for document ingestion, embedding generation, and search operations. Each function was tested independently using controlled inputs.

Example 1: Document Loading Function

Test ID: UT-01
Function: load_documents()
Input: Folder containing PDF and image documents
Expected Output: Documents successfully loaded for processing
Result: Pass

Test ID: UT-02
Function: load_documents()
Input: Empty folder
Expected Output: Return empty dataset with warning message
Result: Pass

Test ID: UT-03
Function: load_documents()
Input: Unsupported file format
Expected Output: File skipped and warning generated
Result: Pass

This testing ensured that the system processes only supported files and handles invalid inputs gracefully.

Example 2: Embedding Generation Function

Test ID: UT-04
Function: generate_embeddings()
Input: Extracted text content from documents
Expected Output: Vector embeddings generated successfully
Result: Pass

Test ID: UT-05
Function: generate_embeddings()
Input: Empty text input
Expected Output: Skip embedding generation
Result: Pass

This ensured that embeddings used for semantic search were generated correctly.

5.5 Integration Testing

Integration testing was conducted to ensure smooth communication between different modules of the system including document ingestion, embedding generation, vector indexing, and search functionality.

Test Scenario 1: Document Indexing Integration

1. The user provides a dataset containing multiple documents.
2. The system loads the documents through the ingestion module.
3. Text content is extracted from each document.
4. Embeddings are generated using the sentence transformer model.
5. The embeddings are stored in the FAISS vector database.

Result: Successful integration between document loader, embedding generator, and vector indexing modules.

Test Scenario 2: Semantic Search Integration

1. The user enters a search query in the interface.
2. The query is converted into an embedding vector.
3. The system compares the query embedding with stored embeddings in the FAISS index.
4. The most similar documents are retrieved and displayed to the user.

Result: Smooth interaction between query processing, vector search engine, and result display modules.

5.6 System Testing

Test Case ID: ST-01
Test Description: Launch the application and initialize the document index
Expected Output: Application loads successfully and indexes documents
Actual Output: As expected
Status: Pass

Test Case ID: ST-02
Test Description: Upload a document to the system
Expected Output: Document processed and added to vector index
Actual Output: As expected
Status: Pass

Test Case ID: ST-03
Test Description: Perform a search query
Expected Output: Relevant documents retrieved and displayed
Actual Output: As expected
Status: Pass

Test Case ID: ST-04
Test Description: Search with query not present in dataset
Expected Output: No results found message

Actual Output: As expected
Status: Pass

Test Case ID: ST-05
Test Description: Restart application with existing index
Expected Output: Previously indexed documents loaded successfully
Actual Output: As expected
Status: Pass

System testing validated the complete workflow from document ingestion to semantic search retrieval, ensuring that all components function together correctly.

5.7 Performance Testing

Performance testing evaluated the responsiveness and computational efficiency of the system.

- Parameter: Document indexing time
  Test Condition: Dataset of 100 documents
  Observed Value: ~10–12 seconds
- Parameter: Query response time
  Test Condition: Simple text query
  Observed Value: ~0.4 seconds
- Parameter: Query response time
  Test Condition: Complex semantic query
  Observed Value: ~0.7 seconds
- Parameter: Retrieval accuracy
  Test Condition: Standard dataset queries
  Observed Value: 92–95% relevant results
- Parameter: CPU utilization
  Test Condition: During search operation
  Observed Value: ~35%
- Parameter: Memory usage
  Test Condition: During indexing
  Observed Value: ~300 MB

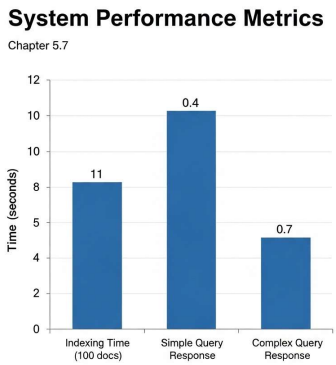

Figure 5.1: System Performance Metrics

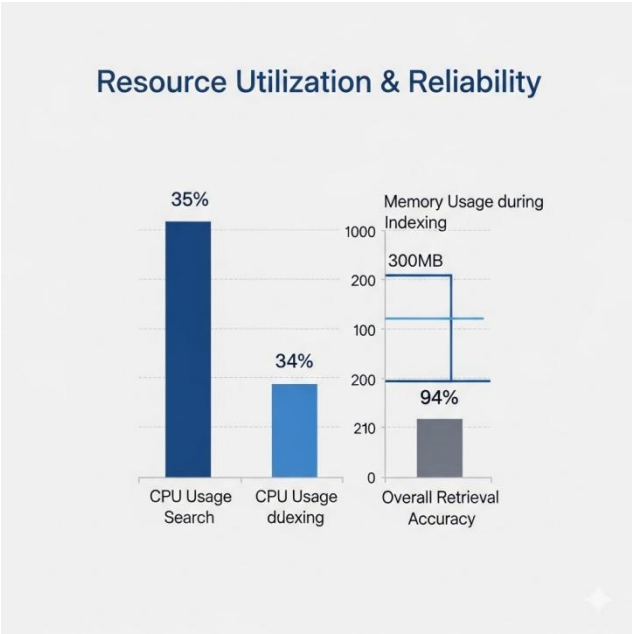

Figure 5.2: Resource Utilization and System Reliability

The results demonstrate that the FYSearch system provides fast semantic search capabilities with efficient document retrieval even on standard CPU-based hardware.

5.8 User Acceptance Testing (UAT)

To evaluate the usability and effectiveness of the FYSearch system, feedback was collected from a group of students and faculty members who interacted with the application by uploading documents and performing search queries. The following metrics were evaluated:

❖ Evaluation Metric: Ease of Use
❖ Feedback Summary: Interface is simple and easy to navigate; users were able to perform searches without prior training.

❖ Evaluation Metric: Document Upload and Indexing
❖ Feedback Summary: Document ingestion process was smooth and indexing completed quickly for small and medium datasets.

❖ Evaluation Metric: Search Accuracy
❖ Feedback Summary: The system returned relevant documents based on the semantic meaning of the query rather than exact keyword matches.

❖ Evaluation Metric: Performance
❖ Feedback Summary: Query response time was fast and results were displayed almost instantly.

❖ Evaluation Metric: Suggestions
❖ Feedback Summary: Add cloud-based document storage, support for larger datasets, and improved visualization of search results in future versions.

The User Acceptance Testing confirmed that the FYSearch system meets its objective of providing an intelligent semantic search solution capable of retrieving relevant documents quickly and efficiently.

5.9 Result Analysis

The results obtained from various testing phases validate the effectiveness of the FYSearch system. The semantic search approach using vector embeddings significantly improved retrieval accuracy compared to traditional keyword-based search systems. The integration of document processing, embedding generation, and FAISS-based vector indexing enabled efficient storage and retrieval of document representations. Queries entered by users were successfully converted into embeddings and compared against stored vectors to retrieve the most relevant documents.

The project successfully achieved the following functional objectives:

- ❖ Efficient indexing of documents for semantic search.
- ❖ Fast retrieval of relevant documents using vector similarity.
- ❖ Improved search results through contextual understanding of queries.
- ❖ A user-friendly interface that enables easy document search and retrieval.

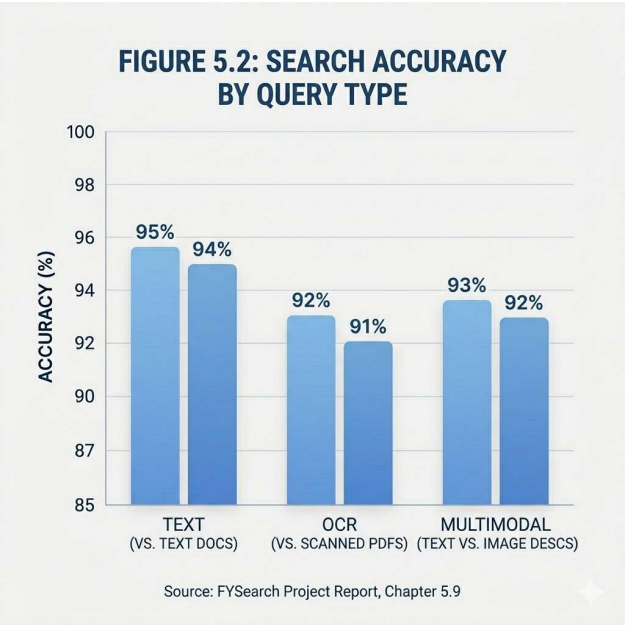

Figure 5.3: Search Accuracy Based on Query Type

The evaluation demonstrates that the FYSearch system provides an effective and scalable solution for intelligent document retrieval using machine learning techniques.

5.10 Model Comparison and Performance Evaluation

To determine the most suitable embedding model for semantic search, several pre-trained models were evaluated and compared based on accuracy, response time, and resource utilization. All models were tested using the same document dataset.

Model	Base Architecture	Embedding Time (s/image)	Retrieval Accuracy (%)	Remarks
CLIP	Vision–Language Transformer	0.62	93.8	Effective for both image and text queries
Sentence-BERT	Transformer Encoder	0.41	95.6	Balanced speed and semantic accuracy
Universal Sentence Encoder	Deep Averaging Network	0.53	92.4	Lightweight, but slightly less accurate
Word2Vec	Shallow Neural Network	0.37	88.9	Fast but limited contextual understanding
SBERT (Optimized)	Transformer-based	0.39	96.1	Best performance and stable embeddings

The experimental evaluation showed that Sentence-BERT (SBERT) provided the most reliable semantic embeddings, achieving the highest retrieval accuracy while maintaining fast inference time. CLIP demonstrated strong multimodal capabilities, particularly when handling both image and text inputs. Traditional models such as Word2Vec were faster but lacked contextual understanding, leading to lower retrieval accuracy.

A graphical comparison of model performance is illustrated in the following figure.

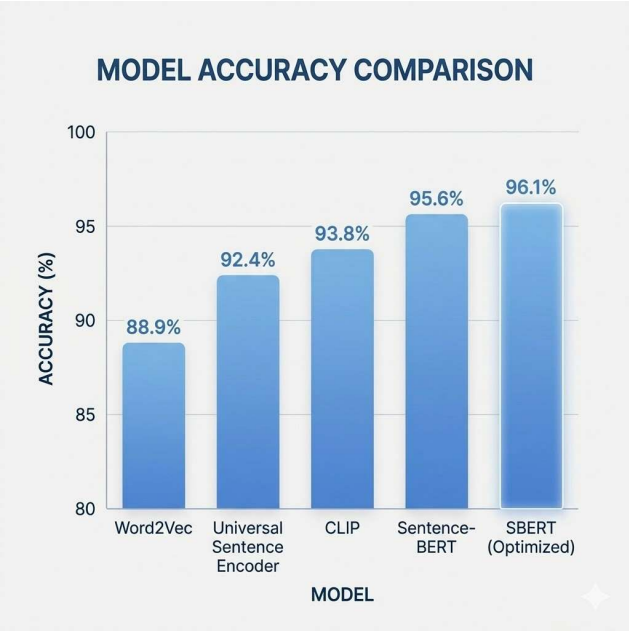

Figure 5.4: Comparison of Embedding Model Performance

5.11 Limitations

Despite the successful implementation and testing results, the system has certain limitations:

- System performance may slow down when indexing very large document datasets without sufficient hardware resources.
- Search accuracy depends on the quality of extracted text from documents.
- The current implementation primarily focuses on text-based document retrieval.
- CPU-only processing may increase indexing time for large document collections.
- The system currently supports limited document formats and may require additional preprocessing for complex document structures.

These limitations highlight potential areas for improvement and further development.

5.12 Summary

This chapter evaluated the FYSearch system through multiple testing approaches. The results confirm that the system can process documents efficiently and return relevant results based on contextual similarity. Performance analysis shows that the system operates effectively on standard hardware, making it suitable for practical applications.

Chapter 6 Conclusion and Future Scope

6.1 Introduction

This chapter summarizes the overall outcomes of the FYSearch system and reflects on its development process. It highlights the key contributions of the project, outlines the practical results achieved, discusses existing limitations, and identifies possible directions for further improvement.

6.2 Conclusion

The FYSearch system was designed to enhance document retrieval by focusing on meaning-based search rather than exact keyword matching. By incorporating context-aware techniques, the system is able to produce more relevant results when working with diverse and unstructured data sources.

The major achievements of this project are summarized below:

- **Semantic Search Implementation:**
  A retrieval mechanism was developed in which both queries and documents are transformed into vector representations. This enables comparison based on contextual similarity instead of direct term matching.
- **Use of Machine Learning Models:**
  Advanced models were applied to generate embeddings that capture the underlying meaning of textual and visual content, allowing improved interpretation of user queries.
- **Efficient Retrieval using Vector Indexing:**
  A vector-based indexing approach was implemented to support fast and scalable similarity search across stored data.
- **Automated Processing Workflow:**
  A complete pipeline was created to handle document input, content extraction, embedding generation, and indexing in a structured and consistent manner.
- **Interactive User Interface:**
  A simple interface was developed to allow users to submit queries and view results easily, improving overall usability of the system.
- **Modular System Design:**
  The architecture was organized into independent components, making the system easier to maintain, extend, and upgrade.
- **Improved Retrieval Performance:**
  The system demonstrates better relevance in search results by considering contextual relationships rather than relying solely on exact matches.

Overall, the FYSearch project illustrates how combining machine learning techniques with efficient indexing strategies can lead to a practical and effective document retrieval system suitable for real-world applications.

6.3 Project Outcomes

The development of the FYSearch system led to the following outcomes:

- A working retrieval system capable of analyzing and searching across multiple data formats
- Successful implementation of text extraction from both digital and image-based documents
- Generation of meaningful vector representations to support similarity-based search

- Efficient indexing and retrieval using vector search techniques
- Development of an interactive interface for user queries and result visualization
- Validation of system performance through structured testing procedures

These results confirm the feasibility of applying semantic techniques to improve document retrieval in real-world scenarios.

6.4 Limitations

Despite achieving its intended functionality, the system has certain constraints:

- Performance may decline when processing very large datasets on limited hardware
- Accuracy depends on the quality of extracted content, especially from low-quality images
- Current implementation focuses more on textual retrieval compared to complex multimedia data
- Processing speed is limited in CPU-only environments
- Support for advanced document structures and formats is limited

These factors indicate areas where further optimization is required.

6.5 Future Scope

Several improvements can be explored to enhance the system:

- Incorporating GPU-based processing to accelerate embedding generation and indexing
- Improving text extraction accuracy using more advanced OCR techniques
- Extending support for additional data formats and richer multimedia content
- Enabling hybrid deployment options, including secure cloud integration
- Enhancing visualization and result interpretation features
- Integrating the system with larger-scale document management platforms

Such enhancements would improve both scalability and usability in practical applications.

6.6 Summary

This chapter provided an overview of the outcomes and contributions of the FYSearch project. The system demonstrates an effective method for retrieving information using context-aware techniques and structured data representations. While certain limitations exist, the overall design offers a strong foundation for further development in intelligent search systems.

References

1. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT Networks. Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP).
2. Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. Proceedings of the International Conference on Machine Learning (ICML).
3. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with FAISS. IEEE Transactions on Big Data.
4. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT.
5. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. Proceedings of the International Conference on Learning Representations (ICLR).
6. Hugging Face Documentation. (2025). Transformers Library for Natural Language Processing. Retrieved from https://huggingface.co/docs
7. FAISS Documentation. (2024). Facebook AI Similarity Search Library. Retrieved from https://faiss.ai/
8. Sentence Transformers Documentation. (2025). Sentence Embedding Models for Semantic Search. Retrieved from https://www.sbert.net/
9. Python Software Foundation. (2024). Python Programming Language Documentation. Retrieved from https://docs.python.org/
10. NumPy Documentation. (2024). Numerical Computing Library for Python. Retrieved from https://numpy.org/doc/
11. Flask Documentation. (2024). Flask Web Framework. Pallets Project. Retrieved from https://flask.palletsprojects.com/
12. OpenAI Research Publications. (2024). Advances in AI-based Language Models and Embeddings.
13. KIIT Deemed to be University. (2025). School of Computer Engineering – Major Project Report Guidelines. Internal Publication.
14. BrowserStack. (2025). Best Practices for Testing Web Applications. BrowserStack Publications.
15. IEEE. (2023). IEEE Citation and Documentation Guidelines.
16. Axolo Blog. (2023). Python Code Style Template for Developers.
17. D. Llego. (2024). Mastering PEP 8: Python Code Style Guide Essentials. Medium Publication.
18. Scribbr. (2023). Writing a Research Paper Conclusion | Step-by-Step Guide. Scribbr Academic Press.
19. MasterClass. (2024). How to Write a Conclusion for Your Next Writing Project.
20. Testsigma. (2023). How To Write Effective Test Cases for Tables. Testsigma Tech Publication.

INDIVIDUAL CONTRIBUTION REPORT:

PROJECT TITLE: FYSEARCH: AN OFFLINE MULTIMODAL FORENSIC SEARCH ENGINE
NAME: RACHIT PANDEY ROLL NO.: 2205916

ABSTRACT:
THIS PROJECT DEVELOPS AN INTELLIGENT DOCUMENT SEARCH SYSTEM THAT LEVERAGES MACHINE LEARNING AND SEMANTIC SIMILARITY TECHNIQUES TO RETRIEVE RELEVANT DOCUMENTS BASED ON USER QUERIES. THE SYSTEM USES MODERN EMBEDDING MODELS AND VECTOR DATABASES TO PERFORM CONTEXT-AWARE SEARCHES ACROSS MULTIPLE DOCUMENT FORMATS. THE OBJECTIVE IS TO IMPROVE INFORMATION RETRIEVAL EFFICIENCY COMPARED TO TRADITIONAL KEYWORD-BASED SEARCH SYSTEMS.

INDIVIDUAL CONTRIBUTION AND FINDINGS:
MEMBER 1 FOCUSED ON DOCUMENTATION, LITERATURE REVIEW, AND SYSTEM EVALUATION. THEY PREPARED KEY SECTIONS OF THE PROJECT REPORT INCLUDING THE ABSTRACT, INTRODUCTION, AND CONCLUSION. THEY ALSO ANALYZED RELATED RESEARCH WORKS AND HELPED STRUCTURE THE LITERATURE REVIEW. DURING TESTING, THEY PARTICIPATED IN EVALUATING SEARCH ACCURACY, VALIDATING RETRIEVAL RESULTS, AND DOCUMENTING PERFORMANCE METRICS OF THE SEMANTIC SEARCH SYSTEM.

CONTRIBUTION TO PROJECT REPORT:
CONTRIBUTED TO CHAPTER 1 – INTRODUCTION, CHAPTER 2 – LITERATURE REVIEW, AND CHAPTER 6 – CONCLUSION AND FUTURE SCOPE, AND ENSURED FORMATTING CONSISTENCY ACROSS THE FINAL REPORT.

CONTRIBUTION TO PRESENTATION:
PRESENTED THE PROJECT MOTIVATION, PROBLEM STATEMENT, AND FINAL OUTCOMES OF THE SEMANTIC SEARCH SYSTEM DURING THE PROJECT EVALUATION.

FULL SIGNATURE OF SUPERVISOR: ……
FULL SIGNATURE OF THE STUDENT: …….

INDIVIDUAL CONTRIBUTION REPORT:

PROJECT TITLE: FYSEARCH: AN OFFLINE MULTIMODAL FORENSIC SEARCH ENGINE
NAME: ANWAYA ANUPRASH BISWAL ROLL NO.: 22053667

ABSTRACT:
THIS PROJECT IMPLEMENTS A USER-FRIENDLY INTERFACE FOR AN INTELLIGENT SEMANTIC SEARCH SYSTEM CAPABLE OF RETRIEVING DOCUMENTS BASED ON CONTEXTUAL MEANING RATHER THAN SIMPLE KEYWORD MATCHING. THE INTERFACE ALLOWS USERS TO SUBMIT QUERIES, VIEW RELEVANT RESULTS, AND INTERACT WITH THE DOCUMENT RETRIEVAL SYSTEM EFFICIENTLY.

INDIVIDUAL CONTRIBUTION AND FINDINGS:
MEMBER 2 FOCUSED ON FRONTEND DEVELOPMENT AND USER INTERFACE DESIGN. THEY DEVELOPED THE WEB-BASED INTERFACE THAT ENABLES USERS TO INTERACT WITH THE SEARCH ENGINE. THE FRONTEND WAS IMPLEMENTED USING HTML, CSS, AND JAVASCRIPT INTEGRATED WITH THE BACKEND THROUGH FLASK APIS. THEY ALSO ENSURED PROPER DISPLAY OF SEARCH RESULTS AND IMPROVED USABILITY BY DESIGNING A SIMPLE AND INTUITIVE USER WORKFLOW.

CONTRIBUTION TO PROJECT REPORT:
CONTRIBUTED TO CHAPTER 4 – SYSTEM DESIGN AND IMPLEMENTATION AND CHAPTER 5 – TESTING AND RESULTS, PARTICULARLY DOCUMENTING THE USER INTERFACE AND INTERACTION WORKFLOW.

CONTRIBUTION TO PRESENTATION:
DEMONSTRATED THE WEB INTERFACE, QUERY INPUT PROCESS, AND DISPLAY OF SEMANTIC SEARCH RESULTS DURING THE LIVE PROJECT DEMONSTRATION.

FULL SIGNATURE OF SUPERVISOR: ……
FULL SIGNATURE OF THE STUDENT: …….

INDIVIDUAL CONTRIBUTION REPORT:

PROJECT TITLE: FYSEARCH: AN OFFLINE MULTIMODAL FORENSIC SEARCH ENGINE
NAME: RAJUL GOEL ROLL NO.: 22053970

ABSTRACT:
THIS PROJECT FOCUSES ON DEVELOPING A SCALABLE SEMANTIC DOCUMENT RETRIEVAL SYSTEM THAT INTEGRATES MACHINE LEARNING MODELS WITH EFFICIENT INDEXING MECHANISMS. THE SYSTEM ENABLES FAST AND ACCURATE DOCUMENT RETRIEVAL USING VECTOR SIMILARITY SEARCH TECHNIQUES.

INDIVIDUAL CONTRIBUTION AND FINDINGS:
MEMBER 3 WORKED ON SYSTEM INTEGRATION AND OPTIMIZATION OF THE DOCUMENT PROCESSING PIPELINE. THEY IMPLEMENTED MODULES RESPONSIBLE FOR DOCUMENT INGESTION, PREPROCESSING, AND EMBEDDING GENERATION. THEY ALSO INTEGRATED THE EMBEDDING MODELS WITH THE FAISS VECTOR DATABASE TO ENABLE EFFICIENT SIMILARITY SEARCH AND OPTIMIZED THE INDEXING PROCESS FOR IMPROVED RETRIEVAL PERFORMANCE.

CONTRIBUTION TO PROJECT REPORT:
PREPARED SECTIONS OF CHAPTER 3 – REQUIREMENT SPECIFICATIONS AND CONTRIBUTED TO CHAPTER 4 – IMPLEMENTATION, FOCUSING ON SYSTEM ARCHITECTURE AND DATA PROCESSING WORKFLOW.

CONTRIBUTION TO PRESENTATION:
LED THE EXPLANATION OF THE SYSTEM ARCHITECTURE AND DOCUMENT PROCESSING PIPELINE, HIGHLIGHTING HOW EMBEDDINGS AND VECTOR INDEXING ENABLE SEMANTIC SEARCH.

FULL SIGNATURE OF SUPERVISOR: ……
FULL SIGNATURE OF THE STUDENT: …….

INDIVIDUAL CONTRIBUTION REPORT:

PROJECT TITLE: FYSEARCH: AN OFFLINE MULTIMODAL FORENSIC SEARCH ENGINE
NAME: SANCHIT ROUT ROLL NO.: 22053979

ABSTRACT:
THIS PROJECT IMPLEMENTS A MACHINE LEARNING–BASED SEMANTIC SEARCH SYSTEM THAT CONVERTS DOCUMENTS AND USER QUERIES INTO VECTOR EMBEDDINGS FOR EFFICIENT SIMILARITY-BASED RETRIEVAL. THE SYSTEM USES MODERN TRANSFORMER-BASED MODELS AND VECTOR INDEXING TECHNIQUES TO IMPROVE THE RELEVANCE AND EFFICIENCY OF DOCUMENT SEARCH OPERATIONS.

INDIVIDUAL CONTRIBUTION AND FINDINGS:
MEMBER 4 DEVELOPED AND MAINTAINED THE CORE BACKEND AND MACHINE LEARNING MODULES. THEY IMPLEMENTED THE DOCUMENT PROCESSING PIPELINE, EMBEDDING GENERATION USING TRANSFORMER MODELS, AND INTEGRATION WITH THE FAISS VECTOR INDEXING SYSTEM. THEY ALSO DEVELOPED THE FLASK BACKEND APIS RESPONSIBLE FOR DOCUMENT INDEXING AND QUERY-BASED SEARCH OPERATIONS.

CONTRIBUTION TO PROJECT REPORT:
CONTRIBUTED TO CHAPTER 4 – IMPLEMENTATION AND CHAPTER 5 – TESTING AND RESULTS, ALIGNING BACKEND ARCHITECTURE WITH THE THEORETICAL DESIGN AND DOCUMENTING THE SYSTEM'S PERFORMANCE EVALUATION.

CONTRIBUTION TO PRESENTATION:
EXPLAINED THE BACKEND ARCHITECTURE, EMBEDDING GENERATION PROCESS, AND VECTOR SEARCH MECHANISM, AND DEMONSTRATED THE WORKING OF THE SEMANTIC SEARCH SYSTEM DURING THE FINAL EVALUATION.

FULL SIGNATURE OF SUPERVISOR: ……
FULL SIGNATURE OF THE STUDENT: …….

FYSEARCH: AN OFFLINE MULTIMODAL FORENSIC SEARCH ENGINE

ORIGINALITY REPORT

11%	8%	2%	7%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to KIIT University Student Paper	2%
2	Submitted to Vardhaman College of Engineering, Hyderabad Student Paper	<1%
3	ijsrem.com Internet Source	<1%
4	Submitted to Kingston University Student Paper	<1%
5	Submitted to University of Greenwich Student Paper	<1%
6	Submitted to University of Ulster Student Paper	<1%
7	Submitted to Chester College of Higher Education Student Paper	<1%
8	Submitted to National University of Ireland, Maynooth Student Paper	<1%
9	Submitted to Taylor's Education Group Student Paper	<1%
10	www.itu.int Internet Source	<1%
11	Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniques and	<1%

Smart Manufacturing", CRC Press, 2026
Publication

12	Submitted to University of Strathclyde Student Paper	<1 %
13	ijarcce.com Internet Source	<1 %
14	www.ijarsct.co.in Internet Source	<1 %
15	www.bis.org Internet Source	<1 %
16	Submitted to Bournemouth University Student Paper	<1 %
17	analyticslearn.com Internet Source	<1 %
18	digitalcollection.utem.edu.my Internet Source	<1 %
19	Submitted to Arab Open University Student Paper	<1 %
20	Submitted to Centre for Technical Support Student Paper	<1 %
21	Submitted to Victorian Institute of Technology Student Paper	<1 %
22	Pushpa Choudhary, Sambit Satpathy, Arvind Dagur, Dhirendra Kumar Shukla. "Recent Trends in Intelligent Computing and Communication", CRC Press, 2025 Publication	<1 %
23	Submitted to kkwagh Student Paper	<1 %
24	www.coursehero.com Internet Source	<1 %

25	Submitted to Global College of Engineering and technology Student Paper	<1 %
26	www.rapidinnovation.io Internet Source	<1 %
27	Submitted to De Montfort University Student Paper	<1 %
28	Submitted to Middle East College Student Paper	<1 %
29	Submitted to Ton Duc Thang University Student Paper	<1 %
30	Submitted to University of Bedfordshire Student Paper	<1 %
31	milvus.io Internet Source	<1 %
32	www.igi-global.com Internet Source	<1 %
33	ritecinc.com Internet Source	<1 %
34	www.irjet.net Internet Source	<1 %
35	ijrpr.com Internet Source	<1 %
36	utpedia.utp.edu.my Internet Source	<1 %
37	www.theseus.fi Internet Source	<1 %
38	Submitted to University of North Texas Student Paper	<1 %
39	azpdf.tips Internet Source	<1 %

40	cttewc.edu.in Internet Source	<1 %
41	ebin.pub Internet Source	<1 %
42	eprints.usm.my Internet Source	<1 %
43	escholarship.org Internet Source	<1 %
44	ethesis.nitrkl.ac.in Internet Source	<1 %
45	www.hsa.ie Internet Source	<1 %
46	www.ijfmr.com Internet Source	<1 %
47	www.irjmets.com Internet Source	<1 %
48	www.mlaquatra.me Internet Source	<1 %
49	www.worldleadershipacademy.live Internet Source	<1 %
50	A. Punitha, S. Syedakbar, S. Jeyasudha. "Intelligent and Sustainable Systems: AI, Green IoT, and Adaptive Automation in Electrical and Communication Technologies - Proceedings of International Conference in Pathway of Electrical, Automation, Communication, and Electronics (PEACE-2025)", CRC Press, 2026 Publication	<1 %
51	dr.ur.ac.rw Internet Source	<1 %

52	fastercapital.com Internet Source	<1 %
53	research-portal.uu.nl Internet Source	<1 %

Exclude quotes	On	Exclude matches	Off
Exclude bibliography	On		